Knowledge-Gap Action Decisions for LLM QA

An experiment on answer / ask / retrieve / abstain / challenge decisions

Dayi Yao 2022011550

Tsinghua University

June 9, 2026

Talk Plan

- Problem: why action selection matters before generation
- Project design: data schema, labels, and evaluation targets
- Engineering implementation: code structure and main functions
- Results: method comparison, ablations, stability, and errors
- Limitations: what the current experiment proves and what it does not

Problem: Answering Is Only One Possible Action

- Many user questions are not ready for a final answer.
- Missing user constraints call for asking a question.
- Missing or fresh facts call for retrieval.
- Unsupported high-stakes claims call for abstention.
- Contradicted assumptions call for challenging the premise.

The project treats this as a supervised action-decision task.

Task Definition

Inputs

- user query
- dialogue context
- retrieved evidence
- candidate answer

Outputs

- `gap_type`: explanation label
- `gold_action`: operational label
- action set: answer, ask, retrieve, abstain, challenge

Evaluation Target

- Primary classification metric: action macro-F1.
- Safety-sensitive metric: wrong-answer rate.
- Utility combines correctness, wrong-answer penalty, over-refusal, over-asking, and turn cost.
- The experiment evaluates the next action, not the surface quality of the final long answer.

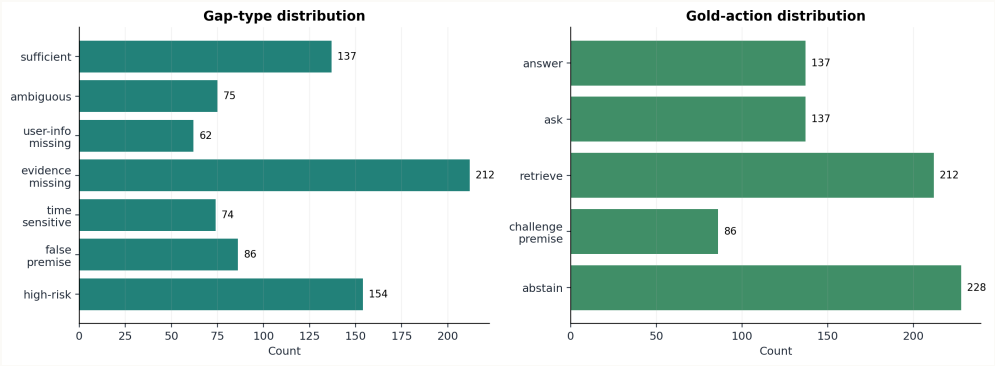
Related Work Used as Background

- AmbigQA and ASQA motivate ambiguous-question and clarification cases.
- FreshLLMs motivates time-sensitive retrieval decisions.
- SelfCheckGPT motivates consistency-style uncertainty features.
- Self-RAG motivates retrieval as a controllable action.
- Sufficient Context and abstention benchmarks motivate evidence-sufficiency and refusal boundaries.

Dataset Overview

- Full run: 800 records generated under controlled knowledge-gap scenarios.
- Split: train 483, validation 156, test 161.
- Split is by `group_id`, so paired variants do not leak across splits.
- DeepSeek generation is cached; fallback templates keep the pipeline runnable offline.

Label Distribution



Record Schema

- `schema.py` defines `GapType`, `Action`, required fields, and validation.
- Key text fields: `user_initial_query`, `dialogue_context`, `retrieved_evidence`, `candidate_answer`.
- Key supervision fields: `gap_type`, `gold_action`, `required_slots`.
- Annotation fields such as risk and evidence sufficiency are recorded, but not directly used as model features.

Controlled Gap Types

- `sufficient_information`: answer is justified.
- `ambiguous_question` and `user_info_missing`: ask before answering.
- `evidence_missing` and `time_sensitive`: retrieve or abstain depending on the claim.
- `false_premise`: correct the assumption first.
- `high_risk_or_expert_needed`: avoid unsupported expert advice.

Repository Structure

- `knowledge_gap_decision/`: core package.
- `data/processed/`: generated dataset, splits, and feature CSVs.
- `results/`: metrics, predictions, ablations, significance tests, and figures.
- `reports/`: generated report and experiment log.
- `tests/`: schema, leakage, feature, and metric checks.

Data Construction Code

- `build_deepseek_dataset`: calls the JSON-mode DeepSeek client and repairs generated records.
- `build_dataset`: deterministic fallback dataset generator.
- `validate_dataset`: catches missing fields, invalid labels, and malformed lists.
- `split_by_group`: performs leakage-resistant train / val / test split.
- `write_dataset`: writes JSONL splits and label distribution artifacts.

Feature Extraction Code

- `compute_features`: central feature table builder.
- Question-side features: length, time words, condition words, subjective words, entity count, risk keywords.
- Evidence-side features: TF-IDF top similarity, token overlap, coverage ratio, contradiction proxy.
- Model-side features: offline pseudo-sample agreement, answer variance, majority ratio.
- `feature_columns`: toggles evidence and uncertainty features for ablations.

Model Code

- `DualClassifier`: trains separate classifiers for `gap_type` and `gold_action`.
- Implemented model kinds: logistic regression, random forest, and GBDT.
- `TextClassifier`: TF-IDF text-only baseline over query, context, evidence, and candidate answer.
- `prompted_llm_baseline`: asks DeepSeek for strict JSON action predictions.
- `full_method_predict`: random-forest prediction plus conservative safety overrides.

Experiment Orchestration

- `run_experiment.run`: one entry point for dataset creation, features, models, metrics, and plots.
- `_select_full_config`: picks a Full Method variant on validation data.
- `_write_error_analysis`: writes sampled real mistakes with key feature values.
- `_write_seed_stability`: reruns the Full Method on three split seeds.
- `scripts/run_full_experiment.py` and `Makefile` provide command-line wrappers.

Evaluation Code

- `evaluate_prediction`: macro-F1, accuracy, retrieval F1, contradiction accuracy, and utility.
- `utility_components`: wrong-answer rate, over-refusal, over-asking, turn cost.
- `per_class_rows`: class-level precision / recall / F1 table.
- `significance_rows`: paired bootstrap and McNemar tests against Full Method.
- `report.py`: assembles markdown and Word reports from result files.

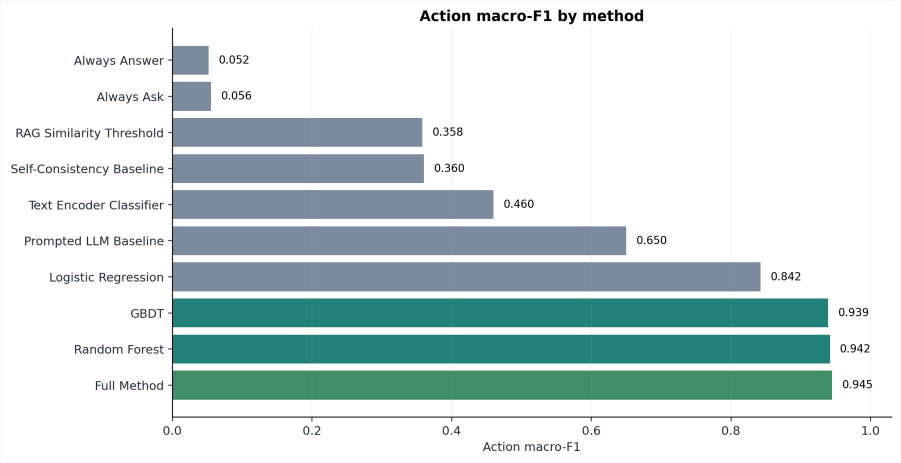
Question Ranker

- `generate_candidates`: creates clarifying questions from missing slots.
- `score_question`: combines slot coverage, uncertainty reduction, specificity, answerability, politeness, and leading-question penalty.
- `rank_questions`: writes `results/question_ranker_eval.csv`.
- The ranker affects the quality of `ask`; it is not the main source of action-F1 gains.

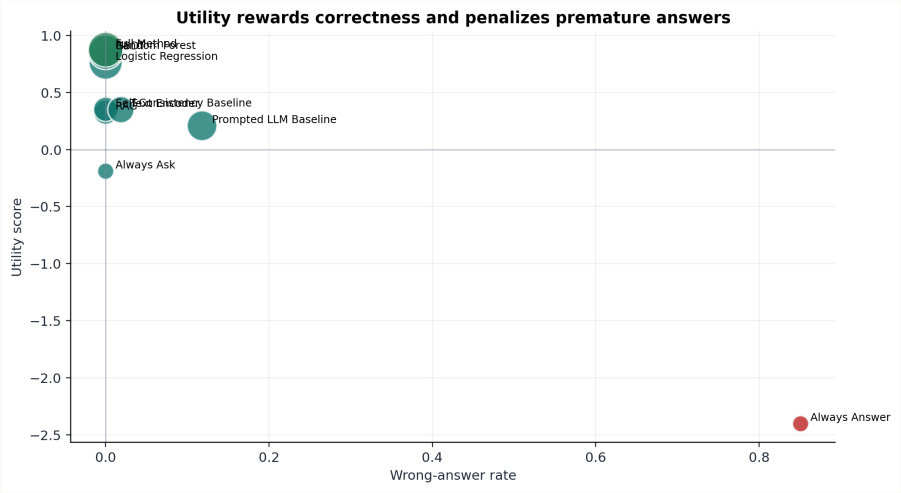
Implementation Caveat

- Current self-consistency features are generated from offline pseudo samples.
- The pseudo-sample generator branches on `gap_type`; this makes it a controlled proxy, not a deployable uncertainty estimator.
- The ablation is still useful for showing that uncertainty-style signals can separate this dataset.
- It should not be described as real black-box LLM self-consistency.

Main Result: Action Macro-F1



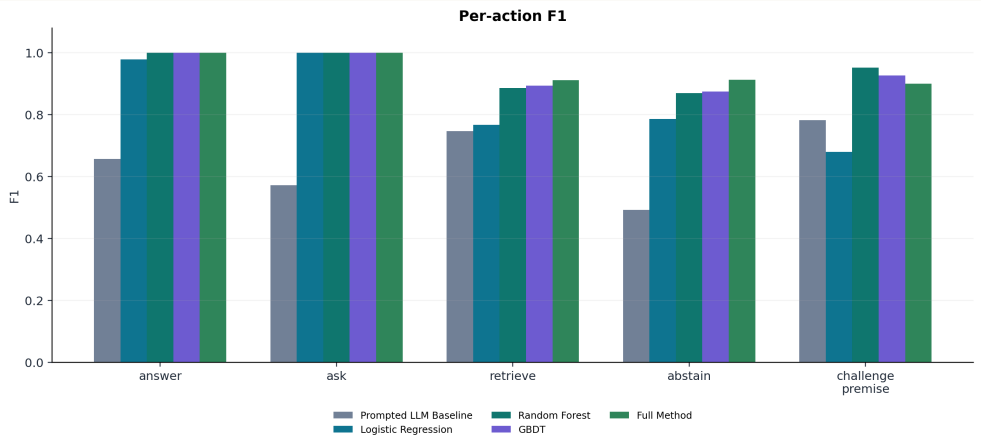
Utility and Wrong-answer Risk



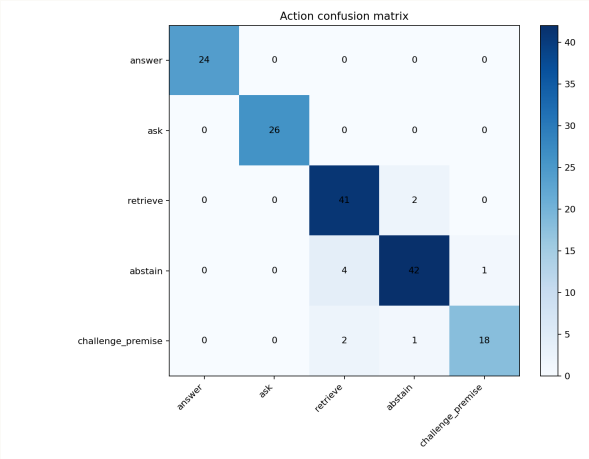
Headline Numbers

- Full Method: action macro-F1 0.945, action accuracy 0.938.
- Full Method: utility 0.875 and wrong-answer rate 0.000.
- Always Answer: action macro-F1 0.052, utility -2.404, wrong-answer rate 0.851.
- Prompted LLM Baseline: action macro-F1 0.650, wrong-answer rate 0.118.

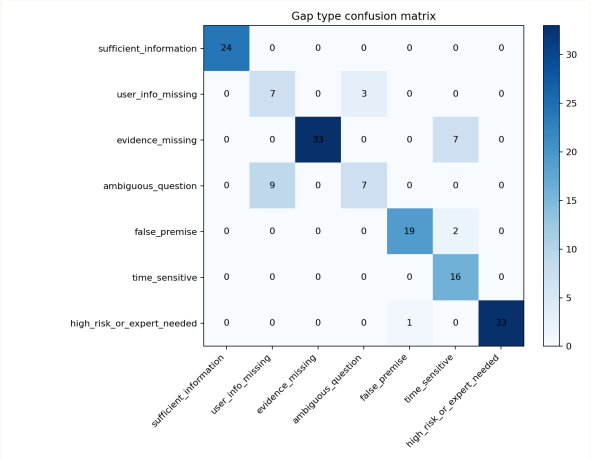
Per-action F1



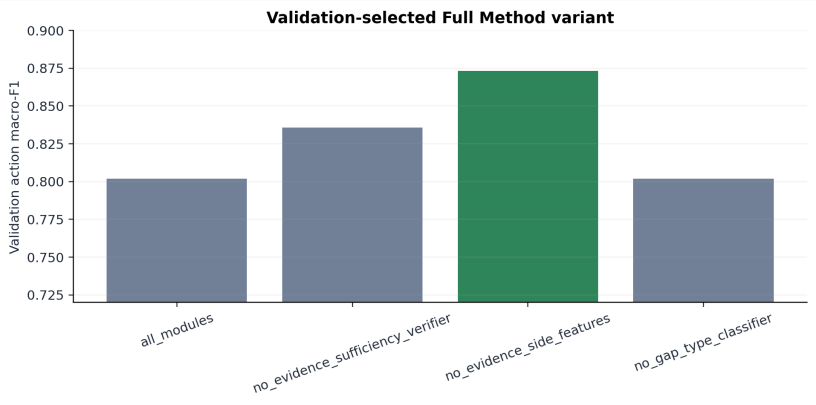
Action Confusion Matrix



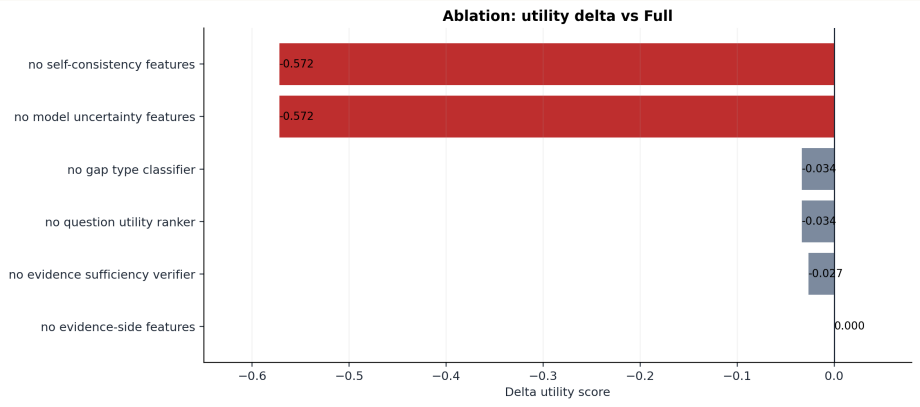
Gap-type Confusion Matrix



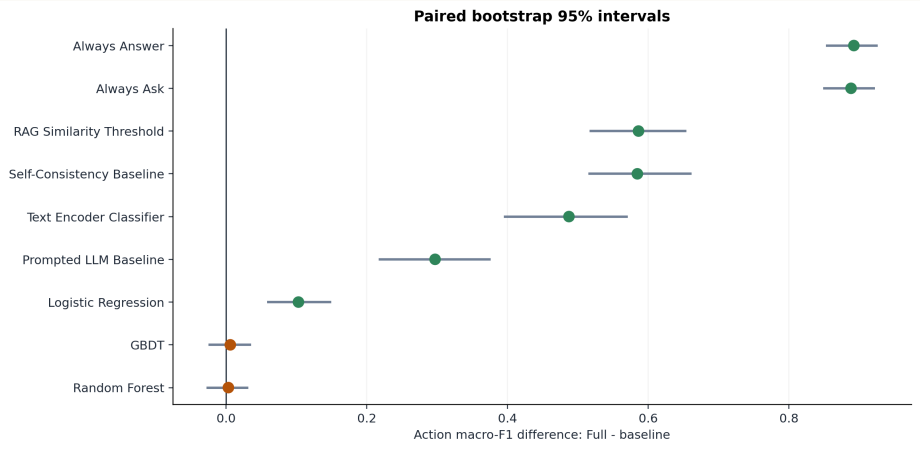
Validation Selection



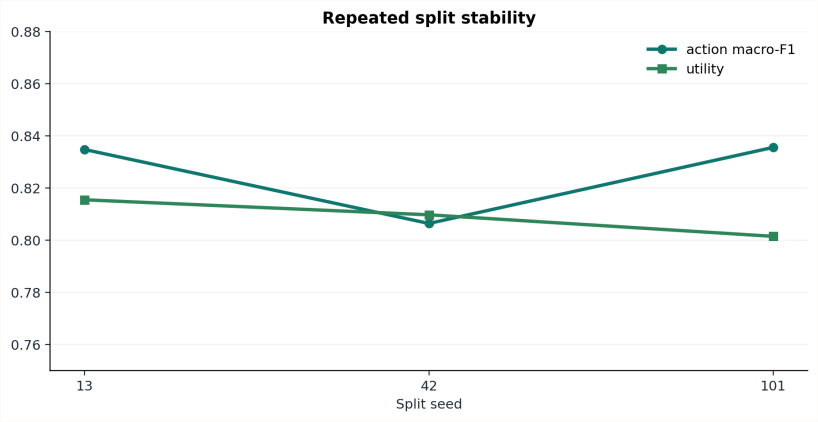
Ablation


Ablation: utility delta vs Full
no self-consistency features
no model uncertainty features
no gap type classifier
no question utility ranker
no evidence sufficiency verifier
no evidence-side features
-0.572
-0.572
-0.034
-0.034
-0.027
0.000
−0.6
−0.5
−0.4
−0.3
−0.2
−0.1
0.0
Delta utility score

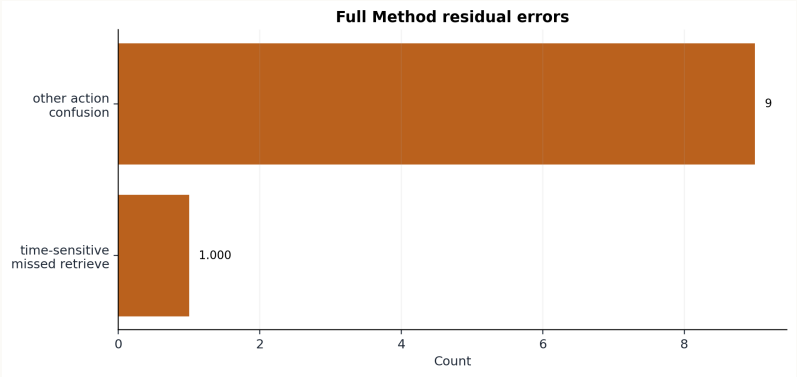
Significance Tests



Repeated Split Stability



Residual Errors



Example Boundaries

- Ambiguous technical question: ask about environment and target before giving steps.
- Current policy or deadline: retrieve fresh evidence before answering.
- False premise: state that the assumption is unsupported before continuing.
- Evidence-support judgment: abstain if the supplied evidence does not support the claim.

What the Results Say

- Explicit action labels are much safer than default answering.
- Structured features beat direct prompted classification on this controlled dataset.
- Full Method is close to Random Forest and GBDT; its advantage over them is not statistically strong.
- The largest ablation drop comes from removing uncertainty-style features.

Limitations

- The dataset is generated under controlled scenarios, not collected from real users.
- Some labels may still reflect the generator's style and scenario design.
- Evidence contradiction is handled by a heuristic proxy, not a strong NLI verifier.
- The current uncertainty feature is not real multi-sample LLM uncertainty.

Next Steps

- Add real user questions and human label adjudication.
- Replace offline pseudo-consistency with real multi-sample uncertainty or calibrated model confidence.
- Add harder adjacent-label examples, especially retrieve vs abstain and false premise vs time-sensitive.
- Test out-of-distribution domains and generation models.

Takeaway

A reliable QA system should decide whether answering is justified before it writes the answer.

This project provides a reproducible prototype for modeling that decision, with clear evidence and clear limits.

References

- Min et al. AmbigQA: Answering Ambiguous Open-domain Questions. EMNLP 2020. arXiv:2004.10645
- Stelmakh et al. ASQA: Factoid Questions Meet Long-Form Answers. EMNLP 2022. arXiv:2204.06092
- Vu et al. FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation. 2023. arXiv:2310.03214
- Manakul et al. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection. 2023. arXiv:2303.08896
- Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. 2023. arXiv:2310.11511
- Sufficient Context: A New Lens on Retrieval Augmented Generation Systems. 2024. arXiv:2411.06037
- Kirichenko et al. AbstentionBench: Reasoning LLMs Fail on Unanswerable Questions. 2025. arXiv:2506.09038